

DCIT208 – Software Engineering

IA1 – Personal System Request-Flow Diagram

Course Code: DCIT208

Assignment Number: IA1

Assignment Title: Personal System Request-Flow Diagram

Full Name: Patience Owusu

Student ID: 22398703

Team Name: TEAM NYLA

Project Title: Nyla Digital Banking Solution

Team Repository URL: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-53-teamnyla.git>

Individual Portfolio Repository URL:

<https://github.com/opashay/DCIT208-Individual-Portfolio.git>

Date: 26 June 2026

Unlisted YouTube Video Link:

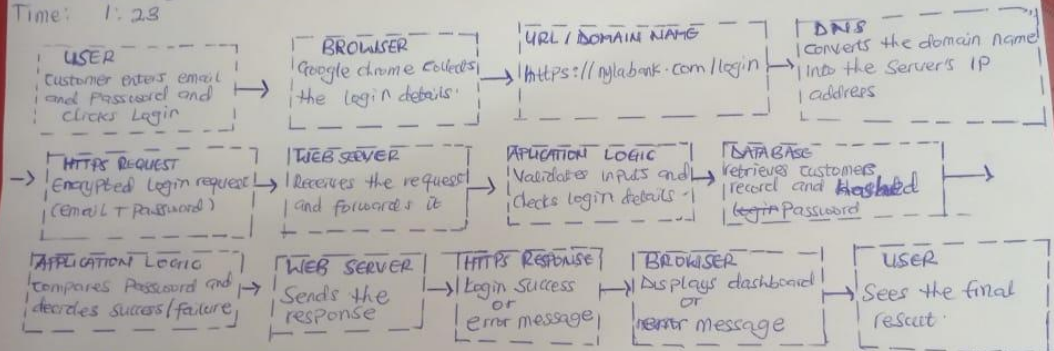
<https://youtube.com/shorts/zFuxR9OOeBg?si=ImNklrlbKy5gZwpb>

AI Disclosure Statement

I used AI assistance to improve my understanding of the software engineering concepts involved in this assignment, including the request flow, system components, and explanation structure. AI-generated suggestions were reviewed, verified against the course materials and project requirements, and adapted to reflect my team's Nyla Digital Banking Solution. I understand that I must be able to explain and defend every part of this submission during a viva if required.

Handwritten Request-Flow Diagram

Name: Patience Owusu
 Student ID: 22398703
 Team: TEAM NYLA
 Project title: Nyla Digital Banking Solution
 Feature: Customer Login
 Date: 26th June 2026
 Time: 1:23



Possible failure Point

The database may be unavailable, which will prevent the application from verifying the customer's login credentials.

Security / Privacy concern

User Passwords must be encrypted during transmission using HTTPS and stored securely as hashed password in the database.

Typed Explanation

Introduction

This request-flow explanation describes how the customer login feature works in the Nyla Digital Banking Solution. The feature was chosen because secure user authentication is one of the most important functions of any digital banking system. The explanation follows the complete path of a login request, from the moment the customer enters their credentials until the system returns a response. Where the project has not yet been implemented, the explanation describes the team's current plan for the system architecture using HTML, CSS, PHP, and MySQL.

Question 1: What device is the user on?

For this request flow, the user is assumed to be using a laptop or desktop computer to access the Nyla Digital Banking Solution. A computer provides a convenient way for customers to access banking services through a web browser. Although the system may later support mobile devices, the current request flow focuses on a customer using a computer to log into their online banking account securely.

Question 2: Is it a browser or mobile app?

The request flow assumes that the customer accesses the Nyla Digital Banking Solution through a web browser, such as Google Chrome or Microsoft Edge. The team currently plans to develop the user interface using HTML and CSS, making the browser the primary platform for accessing the system. If the project later expands to include a mobile application, the request flow will be updated to reflect those changes.

Question 3: What URL, route, screen or endpoint is involved?

The customer accesses the login page using a URL such as <https://nylabank.com/login>. Although this URL is currently used as an example because the system has not yet been deployed, it represents the login page where customers enter their email or account number and password. The URL directs the browser to the correct location so that the login request can be sent to the Nyla Bank server for processing.

Question 4: What protocol is used?

The login feature uses the HTTPS (HyperText Transfer Protocol Secure) protocol to transmit information between the customer's browser and the web server. HTTPS encrypts sensitive information, such as login credentials, while it is being transmitted across the internet. This is particularly important for a banking system because it protects customer information from unauthorised access and helps maintain confidentiality and trust.

Question 5: What data is sent?

When the customer clicks the Login button, the browser sends an HTTPS request containing the login credentials entered by the user. These credentials typically include

the customer's email address or account number and password. The request may also contain technical information, such as the request method and browser details, to help the server process the request correctly. Because the information is transmitted using HTTPS, the data is encrypted while travelling between the browser and the web server.

Question 6: What validates the request?

After the web server receives the login request, it forwards it to the application logic for validation. The application logic first checks whether all required fields have been completed and whether the information is in the correct format. It then retrieves the corresponding customer record from the database and compares the entered password with the securely stored hashed password. If the credentials match, the login request is accepted; otherwise, it is rejected.

Question 7: What database table, collection or external service is touched?

The login feature interacts with the customer records stored in the system's MySQL database. During authentication, the application logic retrieves the customer's account information, including the email or account number and the hashed password. At this stage of the project, the team plans to use MySQL as the database management system because it provides a reliable way to store and retrieve customer information. If the technology stack changes later, this part of the system design will be updated accordingly.

Question 8: What response comes back?

After the application logic completes the authentication process, it sends the result back to the web server, which then returns an HTTPS response to the browser. If the login is successful, the customer is redirected to the banking dashboard, where they can access available services. If the login fails because of incorrect credentials, the browser displays an error message such as "Invalid email or password" and prompts the customer to try again.

Question 9: What happens if validation fails?

If the user enters incorrect or incomplete login details (for example, a missing email format or an empty password), the application logic stops the process before contacting the database. The system performs input validation checks first, and if the data does not meet the required format, the request is rejected immediately. The web server then sends a response back to the browser, prompting the user to correct the input. An error message such as "Please enter a valid email and password" is displayed.

Question 10: What happens if the database is down?

If the MySQL database is unavailable or experiencing downtime, the application logic will not be able to retrieve or verify customer credentials. In this case, the system

returns an error response to the web server, which then sends an HTTPS error message to the browser. The user may see a message like “Service temporarily unavailable, please try again later.” This ensures the system fails gracefully without exposing internal technical details.

Question 11: What happens if 500 users try to log in at once?

If 500 users attempt to log in simultaneously, the web server and database may experience high load. The system handles this using server concurrency mechanisms such as request queuing and thread management. The database processes multiple authentication queries, but performance may slow down depending on capacity. In a well-designed system, load balancing or connection pooling helps distribute requests efficiently to prevent crashes or major delays.

Question 12: What security or privacy concern exists?

A key security concern in the login process is the protection of sensitive user credentials. Passwords must never be stored in plain text; instead, they are hashed and compared securely during authentication. Another concern is preventing unauthorised access through attacks such as brute force login attempts or SQL injection. To reduce these risks, the system should use HTTPS encryption, input sanitisation, and account lockout mechanisms after repeated failed attempts.